

FRONTEND INTEGRATION CONTRACT

Closeté Admin Dashboard API Integration Guide

A source-audited, developer-ready reference for authentication, users, products, orders, payouts, and issue operations.

API version: v1

Base path: {API_ORIGIN}/api/v1

Document version: 1.0 · 30 July 2026

Source of truth: current backend routes, validators, controllers, services, models, and middleware

Contents

1	Quick start and API conventions
2	Authentication and account recovery
3	Admin profile and user directory
4	Product catalog and moderation
5	Orders, workflow, and payouts
6	Issue management
7	Shared models, errors, and implementation checklist

Scope. This guide documents every endpoint needed by the current admin dashboard. Buyer checkout, wishlists, AI analysis, OAuth, Stripe Connect onboarding, and Stripe webhooks are outside the admin UI contract. No aggregate dashboard/stats endpoint exists in the backend.

1. Quick start and API conventions

1.1 Base URL and headers

```
Base URL: https://your-api-host.example/api/v1
Accept: application/json
Content-Type: application/json
Authorization: Bearer <accessToken>
```

All protected admin routes accept a JWT access token in the HTTP `Authorization` header. Role claims must be exactly `ADMIN` or `SUPER_ADMIN`. These two roles currently have identical route permissions.

Token storage. The API returns tokens in JSON and does not set an `HttpOnly` authentication cookie. If a same-origin backend-for-frontend is available, keep tokens in secure `HttpOnly` cookies there. Otherwise keep the access token in memory and protect refresh-token storage carefully.

1.2 Success envelope

```
{
  "success": true,
  "message": "Human-readable message",
  "pagination": { "total": 42, "limit": 10, "page": 1, "totalPage": 5 },
  "data": {}
}
```

message, pagination, and data may be omitted by JSON serialization when undefined. List endpoints in this guide return the row array directly in data; they do **not** wrap it in { result, meta }.

1.3 Error envelope

```
{
  "success": false,
  "message": "Validation Error",
  "errorMessages": [
    { "path": "email", "message": "Invalid email" }
  ]
}
```

In non-production environments, an additional stack property may appear. A route miss returns 404 with the same general shape and an errorMessages[0].path equal to the requested URL.

1.4 Common status codes

Code	Meaning and frontend action
200 OK	Successful read, update, login, or delete.
201 Created	Product or issue created.
400 Bad Request	Schema validation or business-rule failure. Render errorMessages and refresh stale workflow data when relevant.
401 Unauthorized	Missing, invalid, or expired token. Try one refresh; then return to sign-in.
403 Forbidden	Authenticated role lacks permission.
404 Not Found	Requested product, order, issue, or route does not exist.
409 Conflict	External/state conflict, notably unavailable Stripe payout onboarding.
413 Payload Too Large	Upload exceeds configured maximum.

429 Too Many Requests	Rate limit reached. Respect RateLimit/Retry-After response headers.
500 Internal Server Error	Unexpected failure or failed third-party operation. Do not assume a mutation was rolled back; refetch the resource.

1.5 Pagination, filtering, sorting, and projection

Query	Behavior	Supported by
page	Positive numeric page; backend default 1. Invalid/zero values fall back to 1.	Users, products, admin orders
limit	Numeric page size; backend default 10. No explicit maximum is enforced.	Users, products, admin orders
sort	Mongoose sort expression. Default -createdAt; examples: price, -price, name.	Users, products, admin orders
searchTerm	Case-insensitive regex search. Users: name/email/contact. Products: name/brand/description.	Users, products only
fields	Comma-separated projection such as name,email,status. Default excludes __v.	Users, products only
Other keys	Exact Mongo field filters. Useful examples: role=USER, status=active, status=verification, payoutStatus=failed.	Users, products, admin orders

Do not pass UI-only query keys. Unknown keys on filter-enabled endpoints become database equality filters and may produce an empty list. Remove values such as `tab`, `view`, or `search`; the search key is specifically `searchTerm`.

Issues are currently returned as one unpaginated, newest-first array; query parameters are ignored.

1.6 Global rate limits

Scope	Limit	Window
All /api/* routes	300 requests per IP	15 minutes
Password login, recovery, email verification	10 requests per IP	15 minutes

2. Authentication and account recovery

POST /auth/admin/login

Purpose: Password sign-in for ADMIN and SUPER_ADMIN accounts.

Authentication: Public. Rate limited to 10 attempts/IP/15 minutes.

Body field	Type	Required	Description / validation
email	string	Yes	Valid email syntax. The lookup uses the submitted value; normalize to lowercase in the client.
password	string	Yes	No route-level minimum; stored passwords use a six-character model minimum.

Sample request

```
POST /api/v1/auth/admin/login
Content-Type: application/json

{ "email": "admin@example.com", "password": "correct-horse-battery-staple" }
```

200 response

```
{
  "success": true,
  "message": "Admin logged in successfully.",
  "data": {
    "accessToken": "<jwt>",
    "refreshToken": "<jwt>"
  }
}
```

Error	When
400	Account absent, unverified, banned, or password incorrect.
403	The credentials belong to a USER; password login is admin-only.
429	Login rate limit exceeded.

After login, call GET /user/profile. The token response does not include name, image, or account details.

POST /auth/refresh-token

Purpose: Exchange a valid refresh token for a new access token.

Authentication: Public endpoint; refresh token is in the JSON body, not the Authorization header.

Body field	Type	Required	Description
refreshToken	string	Yes	JWT returned by login.

```
POST /api/v1/auth/refresh-token
Content-Type: application/json

{ "refreshToken": "<refresh-jwt>" }
```

200 response

```
{
  "success": true,
  "message": "Access token generated successfully",
  "data": { "accessToken": "<new-access-jwt>" }
}
```

Errors: 400 missing body field; 401 invalid/expired token, deleted user, banned account, or unverified account. The refresh token is not rotated.

POST /auth/change-password

Purpose: Change the signed-in admin's password.

Authentication: Bearer access token. Route permits ADMIN and USER, but currently omits SUPER_ADMIN.

Body field	Type	Required	Rules
currentPassword	string	Yes	Must match stored password.

newPassword	string	Yes	Must differ from current password.
confirmPassword	string	Yes	Must equal newPassword.

```
POST /api/v1/auth/change-password
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "currentPassword": "old-password",
  "newPassword": "new-long-password",
  "confirmPassword": "new-long-password"
}
```

200 response

```
{ "success": true, "message": "Your password has been successfully changed" }
```

Errors: 400 validation, user absent, incorrect current password, password reused, or confirmation mismatch; 401 invalid token; 403 role denied.

Backend caveat: A SUPER_ADMIN token receives 403 on this route even though it can use other admin routes.

2.1 Password-recovery sequence

The recovery flow is a three-request sequence. The six-digit OTP expires after 3 minutes; the reset token returned after OTP verification expires after 5 minutes and is single-use.

POST /auth/forget-password

Purpose: Email a password-reset OTP. Public; 10 requests/IP/15 minutes.

Body: { "email": "admin@example.com" } where email is required and must be valid.

200 response

```
{
  "success": true,
  "message": "Please check your email. We have sent you a one-time passcode (OTP)."
```

Errors: 400 invalid email or user absent; 429 rate limit.

POST /auth/verify-email

Purpose: Verify the reset OTP and obtain a one-time reset token. This endpoint also serves account email verification.

Body field	Type	Required	Description
email	string	Yes	Valid email.
oneTimeCode	number	Yes	Numeric OTP; send as a JSON number, not a string.

```
POST /api/v1/auth/verify-email
Content-Type: application/json

{ "email": "admin@example.com", "oneTimeCode": 123456 }
```

200 reset-flow response

```
{
  "success": true,
  "message": "Verification Successful: Please securely store and utilize this code for reset password",
  "data": "<one-time-reset-token>"
}
```

If the account was not yet verified, the same request verifies it and returns no reset token. Errors are 400 for missing/incorrect/expired OTP or absent user, and 429 for the auth rate limit.

POST /auth/reset-password

Purpose: Set a new password using the reset token.

Authentication: Authorization: Bearer <one-time-reset-token>. This is not an access JWT.

Body field	Type	Required	Rules
newPassword	string	Yes	Confirmation must match.
confirmPassword	string	Yes	Must equal new password.

```
POST /api/v1/auth/reset-password
Authorization: Bearer <one-time-reset-token>
Content-Type: application/json

{ "newPassword": "new-long-password", "confirmPassword": "new-long-password" }
```

200 response

```
{ "success": true, "message": "Your password has been successfully reset." }
```

Errors: 400 missing fields or mismatch; 401 missing, invalid, expired, already-consumed, or unauthorized reset token.

3. Admin profile and user directory

GET /user/profile

Purpose: Load the signed-in admin identity after authentication.

Authentication: Bearer token; ADMIN, SUPER_ADMIN, or USER.

```
GET /api/v1/user/profile
Authorization: Bearer <accessToken>
```

200 response

```
{
  "success": true,
  "message": "Profile data retrieved successfully",
  "data": {
    "_id": "66a0...001",
    "name": "Operations Admin",
    "role": "ADMIN",
    "email": "admin@example.com",
    "contact": "+971500000000",
    "location": "Dubai",
    "image": "https://cdn.example.com/profile.png",
    "provider": "local",
    "status": "active",
    "verified": true,
    "createdAt": "2026-07-01T10:00:00.000Z",
    "updatedAt": "2026-07-20T12:00:00.000Z"
  }
}
```

Errors: 400 user record no longer exists; 401 invalid token; 403 unsupported role.

PATCH /user/profile

Purpose: Update the signed-in admin's basic profile and optional image.

Authentication: Bearer token; all known roles.

Content type: multipart/form-data. Send a JSON string in field data and optionally one file in field image.

data JSON field	Type	Required	Description
name	string	No	Updated display name.
email	string	No	No email-format validation on this update route.
password	string	No	Avoid this field; use change-password so hashing and current-password checks apply.
image	string	No	Normally supplied as the file part instead.

```
const form = new FormData();
form.append("data", JSON.stringify({ name: "Ops Lead" }));
form.append("image", file);

await fetch(`${baseUrl}/user/profile`, {
  method: "PATCH",
  headers: { Authorization: `Bearer ${accessToken}` },
  body: form
}); // Do not manually set Content-Type; the browser adds the boundary.
```

200 response

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "_id": "66a0...001", "name": "Ops Lead", "role": "ADMIN", "...": "..." }
}
```

File rules: JPEG/JPG/PNG only; MIME type and magic bytes are checked. Maximum size is deployment-configured (MAX_UPLOAD_BYTES, default 10 MiB). Up to 3 image parts are technically accepted, but only the first is used.

Errors: 400 malformed data, unsupported/invalid file, or absent user; 401/403 auth; 413 upload too large.

GET /user

Purpose: Paginated user directory for operations and customer support.

Authentication: Bearer token; ADMIN or SUPER_ADMIN.

Query: page, limit, sort, searchTerm, fields, plus exact filters such as role=USER, status=active|ban, verified=true|false, or provider=local|google.

```
GET /api/v1/user?page=1&limit=20&role=USER&status=active&searchTerm=sara&sort=-createdAt
Authorization: Bearer <accessToken>
```

200 response

```
{
  "success": true,
  "message": "Users retrieved successfully",
  "pagination": { "total": 1, "limit": 20, "page": 1, "totalPage": 1 },
  "data": [{
    "_id": "66b0...009",
    "name": "Sara Ali",
    "role": "USER",
    "email": "sara@example.com",
    "contact": "+971501234567",
    "location": "Dubai",
    "image": "https://cdn.example.com/profile.png",
    "provider": "local",
    "status": "active",
    "verified": true,
    "createdAt": "2026-07-25T09:00:00.000Z",
    "updatedAt": "2026-07-25T09:05:00.000Z"
  }]
}
```

Errors: 401/403 auth; 500 malformed Mongo query/sort/projection. Sensitive password, OTP, authentication, login-OTP, and Stripe account fields are excluded by model selection.

Capability gap: There is no admin endpoint to view an arbitrary user by ID, ban/unban users, change roles, create admin accounts, or delete another user. The public POST /user creates only a USER, so it is not an admin-management API.

4. Product catalog and moderation

GET /products

Purpose: Browse the catalog for moderation. Public endpoint; no token required.

Query: Standard pagination/sort/search/projection plus exact filters. If `status` is omitted, the backend forces `status=available`. Admin views must explicitly send `status=secured` or `status=sold` to see those states.

```
GET /api/v1/products?page=1&limit=20&status=secured&searchTerm=chanel&sort=-createdAt
```

200 response

```
{
  "success": true,
  "message": "Products retrieved successfully",
  "pagination": { "total": 1, "limit": 20, "page": 1, "totalPage": 1 },
  "data": [{
    "_id": "66c0...110",
    "name": "Classic Flap Bag",
    "image": "https://cdn.example.com/product-images/bag.jpg",
    "brand": "Chanel",
    "description": "Black lambskin, gold hardware",
    "price": 12500,
    "condition": "Excellent",
    "proofOfPurchase": "https://cdn.example.com/product-proofs/receipt.pdf",
    "status": "secured",
    "seller": {
      "_id": "66b0...009", "name": "Sara Ali",
      "location": "Dubai", "contact": "+971501234567"
    },
    "buyer": "66b0...010",
    "orderId": 1042,
    "createdAt": "2026-07-25T09:00:00.000Z",
    "updatedAt": "2026-07-26T09:00:00.000Z"
  }]
}
```

Errors: Usually 500 for invalid query syntax. Product-list results are cached for 60 seconds by the full query and invalidated on product create/update/delete and issue resolution.

GET**/products/:id**

Purpose: Fetch one complete product. Public endpoint.

Path field	Type	Required	Description
id	Mongo ObjectId	Yes	Product _id, not numeric orderId.

```
GET /api/v1/products/66c0f1234567890123456110
```

200: Same product object shape as above in data, with seller populated as name/location/contact.

Errors: 404 product absent; malformed ObjectId may surface as 500 in production.

POST**/products**

Purpose: Create a product listing. Admins may use this route, and become the product's seller.

Authentication: Bearer token; all roles.

Content type: multipart/form-data. Prefer a JSON string in data, one required image file, and an optional doc PDF.

data JSON field	Type	Required	Description / validation
name	string	Yes	Product display name.
brand	string	Yes	Brand label.
description	string	Yes	Product description.
price	number	Yes	JSON number. No explicit positive/minimum validation.
condition	string	Yes	Free-form; no enum is enforced.

```
const form = new FormData();
form.append("data", JSON.stringify({
  name: "Classic Flap Bag",
  brand: "Chanel",
  description: "Black lambskin, gold hardware",
  price: 12500,
  condition: "Excellent"
}));
form.append("image", imageFile); // JPEG or PNG, required
form.append("doc", proofPdf);    // PDF, optional
```

201 response

```
{
  "success": true,
  "message": "Product created successfully",
  "data": {
    "_id": "66c0...110",
    "name": "Classic Flap Bag",
    "image": "https://cdn.example.com/product-images/bag.jpg",
    "brand": "Chanel",
    "description": "Black lambskin, gold hardware",
    "price": 12500,
    "condition": "Excellent",
    "proofOfPurchase": "https://cdn.example.com/product-proofs/receipt.pdf",
    "status": "available",
    "seller": "66a0...001",
    "orderId": 1042,
    "createdAt": "2026-07-30T10:00:00.000Z",
    "updatedAt": "2026-07-30T10:00:00.000Z"
  }
}
```

File rules: image accepts JPEG/JPG/PNG; doc accepts PDF. MIME type and file signature are checked. Deployment maximum defaults to 10 MiB per file; middleware allows at most 6 total files and 3 per field, although this service uses only the first image and first document.

Business logic: Seller comes from the token, status is forced to `available`, and a sequential numeric `orderId` is assigned starting at 1000. Files are uploaded to object storage.

Errors: 400 missing required fields/image, malformed data, unsupported or spoofed file; 401/403 auth; 413 size; 500 storage/database failure.

PATCH /products/:id

Purpose: Edit product metadata or moderation status.

Authentication: Bearer token; all roles. Service authorizes the seller or either admin role.

JSON body field	Type	Required	Validation
name	string	No	Free-form.
brand	string	No	Free-form.
description	string	No	Free-form.
price	number	No	Must be JSON number.
condition	string	No	Free-form.
status	enum	No	available, secured, or sold.

```
PATCH /api/v1/products/66c0f1234567890123456110
Authorization: Bearer <accessToken>
Content-Type: application/json

{ "condition": "Very good", "status": "available" }
```

200: Updated product in data. **Errors:** 400 validation; 401/403 auth/ownership; 404 absent product.

This endpoint does not accept replacement files and does not enforce order workflow. Changing status manually can desynchronize the product from its order; prefer order/issue operations for secured products.

DELETE /products/:id

Purpose: Permanently remove a product and its object-storage image/proof.

Authentication: Bearer token; seller or either admin role.

```
DELETE /api/v1/products/66c0f1234567890123456110
Authorization: Bearer <accessToken>
```


200 response

```
{ "success": true, "message": "Product deleted successfully", "data": null }
```

Errors: 401/403 auth/ownership; 404 absent product; 500 object-storage/database failure.

Deletion is hard and does not check for an associated order. For verification failures, use the issue-resolution flow so refunds/reversals and seller notification happen first.

5. Orders, workflow, and payouts

GET /orders/admin/all

Purpose: Paginated operational order queue.

Authentication: Bearer token; ADMIN or SUPER_ADMIN.

Query: page, limit, sort, plus exact equality filters. There is no search or projection on this endpoint. Useful filters: status, payoutStatus, payment.status, buyer, seller, or product.

```
GET /api/v1/orders/admin/all?page=1&limit=20&status=verification&sort=-createdAt
Authorization: Bearer <accessToken>
```

200 response

```
{
  "success": true,
  "message": "Orders retrieved successfully",
  "pagination": { "total": 1, "limit": 20, "page": 1, "totalPage": 1 },
  "data": [{
    "_id": "66d0...220",
    "orderNumber": "CLT-12345678901",
    "product": { "_id": "66c0...110", "name": "Classic Flap Bag", "...": "..." },
    "buyer": {
      "_id": "66b0...010", "name": "Buyer Name", "email": "buyer@example.com",
      "contact": "+971500000001", "location": "Dubai"
    },
    "seller": {
      "_id": "66b0...009", "name": "Sara Ali", "email": "sara@example.com",
      "contact": "+971501234567", "location": "Dubai"
    },
    "price": 12500,
    "platformFee": 1500,
    "sellerPayout": 11000,
    "deliveryDetails": {
      "address": "Apartment 5", "location": "Dubai Marina", "phone": "+971500000001"
    },
    "payment": { "provider": "stripe", "paymentIntentId": "pi_...", "status": "paid" },
    "payoutStatus": "pending",
    "status": "verification",
    "statusHistory": [{
      "status": "verification",
      "note": "Received at authentication center",
      "changedAt": "2026-07-29T08:00:00.000Z",
      "changedBy": "66a0...001"
    }],
    "createdAt": "2026-07-26T08:00:00.000Z",
    "updatedAt": "2026-07-29T08:00:00.000Z"
  }]
}
```

Errors: 401/403 auth; malformed query expressions may become 500.

GET

/orders/:id

Purpose: Fetch a full order and populated participants for a detail drawer/page.

Authentication: Bearer token; admins or an order participant.

Path field	Type	Required	Description
------------	------	----------	-------------

id	Mongo ObjectId	Yes	Order document _id, not orderNumber.
----	----------------	-----	--------------------------------------

200: One order object in data, shaped like the list sample. **Errors:** 401/403 auth; 404 absent order; malformed ID may surface as 500.

5.1 Order status state machine

Current status	Permitted next status(es)	Side effects
pending_payment	secured	Normally entered automatically by Stripe webhook, not by admin.
secured	collection_pending, cancelled	Cancel refunds paid payment and relists product.
collection_pending	collected, cancelled	Cancel refunds and relists.
collected	verification	No external side effect.
verification	payout_processing, refunded	Payout transition sends Stripe transfer; refund relists product.
payout_processing	ready_for_delivery	No new transfer; payout should already be paid.
ready_for_delivery	delivered	Product becomes sold.
delivered	completed	Product remains sold.
completed, refunded, cancelled	None	Terminal states.

PATCH /orders/:id/status

Purpose: Advance an order through the controlled operations workflow.

Authentication: Bearer token; ADMIN or SUPER_ADMIN.

Body field	Type	Required	Description
status	enum	Yes	One of the 11 lowercase order statuses shown in the state machine.
note	string	No	Operations note appended to status history.

```
PATCH /api/v1/orders/66d0f1234567890123456220/status
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{ "status": "payout_processing", "note": "Authentication passed" }
```

200 response

```
{
  "success": true,
  "message": "Order status updated successfully",
  "data": {
    "_id": "66d0...220",
    "status": "payout_processing",
    "payoutStatus": "paid",
    "payoutTransferId": "tr...",
    "statusHistory": [{
      "status": "payout_processing",
      "note": "Authentication passed",
      "changedAt": "2026-07-30T11:00:00.000Z",
      "changedBy": "66a0...001"
    }],
    "...": "..."
  }
}
```

Errors: 400 invalid enum/transition, unsettled buyer payment, or payout attempted outside verification; 401/403 auth; 404 order absent; 409 seller has no connected Stripe account or onboarding/payout capability is incomplete; 500 Stripe refund/transfer failure.

Payout behavior: Moving `verification` → `payout_processing` immediately calls Stripe. On transfer success, `payoutStatus` becomes `paid`; on failure it becomes `failed`, the order status remains `verification`, and the request errors. Refetch after every failure.

PATCH `/orders/:id/payout`

Purpose: Retry or explicitly initiate the seller Stripe transfer for an order in `verification` or `payout_processing`.

Authentication: Bearer token; ADMIN or SUPER_ADMIN. No request body.

```
PATCH /api/v1/orders/66d0f1234567890123456220/payout
Authorization: Bearer <accessToken>
```

200 response

```
{
  "success": true,
  "message": "Seller payout marked as paid",
  "data": {
    "_id": "66d0...220",
    "status": "verification",
    "payoutStatus": "paid",
    "payoutTransferId": "tr_...",
    "...": "..."
  }
}
```

Validation/business logic: Buyer payment must be paid; order status must be `verification` or `payout_processing`; seller's connected account must have submitted details and enabled payouts. If already paid, the call is idempotent and returns the order without another transfer.

Errors: 400 state/payment ineligible; 401/403 auth; 404 order absent; 409 Stripe account unavailable/incomplete; 500 transfer failure.

Despite the response wording, this endpoint does not merely mark a manual payment. It creates a real Stripe transfer.

6. Issue management

GET /issues

Purpose: Return the complete issue queue, newest first.

Authentication: Bearer token; ADMIN or SUPER_ADMIN.

Pagination/filtering: None. Query parameters are ignored.

```
GET /api/v1/issues
Authorization: Bearer <accessToken>
```

200 response

```
{
  "success": true,
  "message": "Issues retrieved successfully",
  "data": [{
    "_id": "66e0...330",
    "product": { "_id": "66c0...110", "name": "Classic Flap Bag", "...": "..." },
    "buyer": { "_id": "66b0...010", "name": "Buyer Name", "...": "..." },
    "seller": { "_id": "66b0...009", "name": "Sara Ali", "...": "..." },
    "issueType": "verification_failed",
    "reason": "Serial number could not be authenticated",
    "admin": { "_id": "66a0...001", "name": "Operations Admin", "...": "..." },
    "resolved": false,
    "createdAt": "2026-07-30T12:00:00.000Z",
    "updatedAt": "2026-07-30T12:00:00.000Z"
  }]
}
```

Errors: 401/403 auth; 500 database/population failure. Populated user objects use default model selection and may contain normal profile/contact fields.

GET /issues/:id

Purpose: Fetch one fully populated issue.

Authentication: Bearer token; ADMIN or SUPER_ADMIN.

200: Same object shape as the list sample in data. **Errors:** 401/403 auth; 404 issue absent; malformed ObjectId may surface as 500.

POST /issues

Purpose: Record a buyer refusal or failed product verification and apply financial safeguards.

Authentication: Bearer token; ADMIN or SUPER_ADMIN.

Body field	Type	Required	Description / validation
productId	string	Yes	Product Mongo _id.
issueType	enum	Yes	buyer_refused or verification_failed.
reason	string	Yes	Human-readable evidence/reason.

```
POST /api/v1/issues
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "productId": "66c0f1234567890123456110",
  "issueType": "verification_failed",
  "reason": "Serial number could not be authenticated"
}
```

201 response

```
{
  "success": true,
  "message": "Issue created successfully",
  "data": {
    "_id": "66e0...330",
    "product": "66c0...110",
    "buyer": "66b0...010",
    "seller": "66b0...009",
    "issueType": "verification_failed",
    "reason": "Serial number could not be authenticated",
    "admin": "66a0...001",
    "resolved": false,
    "createdAt": "2026-07-30T12:00:00.000Z",
    "updatedAt": "2026-07-30T12:00:00.000Z"
  }
}
```


Business logic: Only one unresolved issue per product. If a paid order is found, any paid seller transfer is reversed first, then the buyer payment is refunded, the order becomes refunded, and status history is appended. The seller is emailed after the issue is created.

Errors: 400 invalid fields/enum or unresolved duplicate; 401/403 auth; 404 product absent; 500 Stripe/email/database failure.

Email sending is awaited after issue creation. An email failure can make the HTTP request return 500 even though financial actions and/or the issue database write already occurred. Always refetch issues and the order before offering a retry.

PATCH /issues/:id/resolve

Purpose: Close an unresolved issue by deleting or relisting its product.

Authentication: Bearer token; ADMIN or SUPER_ADMIN.

Body field	Type	Required	Description
action	enum	Yes	delete or make_available.

```
PATCH /api/v1/issues/66e0f1234567890123456330/resolve
Authorization: Bearer <accessToken>
Content-Type: application/json

{ "action": "make_available" }
```

200 response

```
{
  "success": true,
  "message": "Issue resolved successfully",
  "data": {
    "_id": "66e0...330",
    "product": { "_id": "66c0...110", "name": "Classic Flap Bag", "...": "..." },
    "issueType": "verification_failed",
    "reason": "Serial number could not be authenticated",
    "resolved": true,
    "...": "..."
  }
}
```

Action behavior: `delete` deletes the product's object-storage files and database record.

`make_available` sets product status to available and clears its buyer. Both email the seller, mark the issue resolved, and clear product-list cache.

Errors: 400 invalid action or already resolved; 401/403 auth; 404 issue/product absent; 500 storage/email/database failure.

The `adminId` passed to the service is not stored as a resolver/audit field. Resolution is not transactionally atomic; refetch after an error.

7. Shared reference

7.1 TypeScript contract starter

```
type UserRole = "SUPER_ADMIN" | "ADMIN" | "USER";
type UserStatus = "active" | "ban";
type ProductStatus = "available" | "secured" | "sold";
type IssueType = "buyer_refused" | "verification_failed";
type IssueResolution = "delete" | "make_available";

type OrderStatus =
  | "pending_payment" | "secured" | "collection_pending" | "collected"
  | "verification" | "payout_processing" | "ready_for_delivery"
  | "delivered" | "completed" | "refunded" | "cancelled";

type PaymentStatus = "pending" | "paid" | "failed" | "refunded";
type PayoutStatus = "pending" | "processing" | "paid" | "failed" | "reversed";

interface ApiSuccess<T> {
  success: true;
  message?: string;
  pagination?: { total: number; limit: number; page: number; totalPage: number };
  data?: T;
}

interface ApiFailure {
  success: false;
  message: string;
  errorMessages?: Array<{ path: string; message: string }>;
  stack?: string;
}
```

7.2 Frontend request policy

1. Attach Bearer <accessToken> to protected requests.
2. On the first 401, run one shared/single-flight refresh request using the refresh token in JSON body.
3. Retry the original request once with the new access token. If refresh fails, clear auth state and return to login.
4. Never automatically retry non-idempotent product, issue, status, or payout mutations after a network/500 response. Refetch first.
5. Debounce user/product search and use the exact searchTerm query key.
6. For multipart requests, let the browser create the Content-Type boundary.

7. Disable workflow actions while pending and refetch order/product/issue data after any mutation.

7.3 Known backend gaps affecting the dashboard

Gap	Frontend handling
No aggregate metrics/stats endpoint	Do not fetch unbounded lists to compute dashboard totals. Request a backend aggregation endpoint before shipping analytics cards/charts.
No admin management, user detail, ban/unban, or role mutation APIs	Keep those controls out of the UI or clearly disabled.
Issues have no pagination/filtering	Filter locally only for small datasets; prioritize a paginated backend endpoint as volume grows.
Order admin list has no text search	Use exact filters and direct detail navigation by document ID. Do not send <code>searchTerm</code> .
Product listing defaults to available	Request each non-available status explicitly; there is no single "all statuses" query.
<code>SUPER_ADMIN</code> omitted from change-password route	Hide that action for super admins until backend permission is corrected.
No transactional boundary around external side effects	After 500/network failures, refetch before retrying payout, issue, and resolution actions.
No documented proof-download endpoint	<code>proofOfPurchase</code> is a returned object-storage URL. Access behavior depends on CDN/bucket policy.

7.4 Integration acceptance checklist

- Login stores both tokens and immediately loads `/user/profile`.
- One refresh request services concurrent 401 responses; each original request retries at most once.
- Admin routing checks uppercase role values exactly.
- Tables consume `data` as the row array and `pagination` as metadata.
- Empty UI filters are removed instead of sent as query parameters.
- Order controls are derived from the state-transition table, not from a free-form status dropdown.
- Payout and issue actions display consequences and require confirmation.
- Mutation errors trigger a resource refetch before a retry is offered.
- Product and profile upload validation mirrors MIME, signature, and configured size constraints.
- Unsupported capabilities listed above are not represented as working controls.

Audit basis. This contract was derived from `src/routes/index.ts`, module `route/validation/controller/service/model` files, authentication and upload middleware, query builder, enums, and the global response/error handlers in the backend repository as of 30 July 2026.